

Lab Assignment 2: Data Wrangling II

Create an “Academic performance” dataset of students and perform the following operations using Python.

STEP 1: Import Required Libraries

```
import pandas as pd
import numpy as np
import scipy
import scipy.stats as stats
import matplotlib.pyplot as plt
import seaborn as sns
```

STEP 2: Load the Academic Performance Dataset and check the structure

```
df=pd.read_csv("/StudentsPerformance.csv")
```

df

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72.0	72.0	74.0
1	female	group C	some college	standard	completed	69.0	90.0	88.0
2	female	group B	master's degree	standard	none	90.0	95.0	93.0
3	male	group A	associate's degree	free/reduced	none	47.0	57.0	NaN
4	male	group C	some college	standard	none	76.0	78.0	75.0
...	...	...	...	...	...	...	...	...
995	female	group E	master's degree	standard	completed	88.0	99.0	95.0
996	male	group C	high school	free/reduced	none	62.0	55.0	55.0
997	female	group C	high school	free/reduced	completed	59.0	71.0	65.0
998	female	group D	some college	standard	completed	68.0	78.0	77.0
999	female	group D	some college	free/reduced	none	77.0	86.0	86.0

Next steps: [Generate code with df](#) [New interactive sheet](#)

df.shape

(1000, 8)

df.columns

Index(['gender', 'race/ethnicity', 'parental level of education', 'lunch', 'test preparation course', 'math score', 'reading score', 'writing score'], dtype='object')

df.dtypes

	0
gender	object
race/ethnicity	object
parental level of education	object
lunch	object
test preparation course	object
math score	float64
reading score	float64
writing score	float64

dtype: object

PART 1: Missing Values & Inconsistencies

1.1 Check missing values

```
df.isnull().sum()
```

	0
gender	0
race/ethnicity	0
parental level of education	0
lunch	0
test preparation course	0
math score	6
reading score	6
writing score	3

dtype: int64

1.2 Fill missing values

```
df['math score'].fillna(df['math score'].median(), inplace=True)
df['reading score'].fillna(df['reading score'].median(), inplace=True)
df['writing score'].fillna(df['writing score'].median(), inplace=True)
```

/tmp/ipython-input-1232232278.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]

```
df['math score'].fillna(df['math score'].median(), inplace=True)
/tmp/ipython-input-1232232278.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]

```
df['reading score'].fillna(df['reading score'].median(), inplace=True)
/tmp/ipython-input-1232232278.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]

```
df['writing score'].fillna(df['writing score'].median(), inplace=True)
```

```
df.isnull().sum()
```

	0
gender	0
race/ethnicity	0
parental level of education	0
lunch	0
test preparation course	0
math score	0
reading score	0
writing score	0

dtype: int64

1.3 Check Inconsistencies in Categorical Columns

```
df['gender'].unique()
df['race/ethnicity'].unique()
df['parental level of education'].unique()
```

```
df['lunch'].unique()
df['test preparation course'].unique()

array(['none', 'completed'], dtype=object)
```

Part 2 : Scan all numeric variables for outliers. If there are outliers, use any of the suitable techniques to deal with them.

2.1 Identify Numeric Columns

```
numeric_cols = ['math score', 'reading score', 'writing score']
```

2.2 Define Z-Score Function

```
def detect_outliers_zscore(data):
    threshold = 3
    mean = np.mean(data)
    std = np.std(data)

    outliers = []

    for value in data:
        z_score = (value - mean) / std
        if abs(z_score) > threshold:
            outliers.append(value)

    return outliers
```

2.3 Detect Outliers

```
for col in numeric_cols:
    outliers = detect_outliers_zscore(df[col])
    print(f"{col} Outliers:", outliers)

math score Outliers: [120.0, 18.0, 10.0, 0.0, 1.0, 200.0, 8.0]
reading score Outliers: [400.0, 134.0, 11.0]
writing score Outliers: [10.0, 350.0, 191.0, 0.0]
```

2.4 Remove Outliers

```
for col in numeric_cols:
    outliers = detect_outliers_zscore(df[col])
    df = df[~df[col].isin(outliers)]
```

2.5 Verify shape after removal

```
df.shape
```

```
(986, 8)
```

```
df[['math score', 'reading score', 'writing score']].describe()
```

	math score	reading score	writing score
count	986.000000	986.000000	986.000000
mean	66.353955	69.425963	68.243408
std	14.819163	14.383616	14.907280
min	19.000000	23.000000	15.000000
25%	57.000000	60.000000	58.000000
50%	66.000000	70.000000	69.000000
75%	77.000000	79.000000	79.000000
max	100.000000	111.000000	100.000000

```
df = df[(df['math score'] <= 100) &
        (df['reading score'] <= 100) &
        (df['writing score'] <= 100)]
```

```
df[['math score', 'reading score', 'writing score']].describe()
```

	math score	reading score	writing score
count	985.000000	985.000000	985.000000
mean	66.344162	69.383756	68.224365
std	14.823498	14.329703	14.902850
min	19.000000	23.000000	15.000000
25%	57.000000	60.000000	58.000000
50%	66.000000	70.000000	69.000000
75%	77.000000	79.000000	79.000000
max	100.000000	100.000000	100.000000



Part 3: Data Transformation (Change the scale: We convert marks from 0–100 → 0–1)

3.1 Import required libraries

```
from sklearn.preprocessing import MinMaxScaler
```

3.2 Apply Min-Max Scaling

```
scaler = MinMaxScaler()

df[['math_scaled', 'reading_scaled', 'writing_scaled']] = scaler.fit_transform(
    df[['math score', 'reading score', 'writing score']]
)

df.head()
```

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score	math_scaled	reading_scaled	writing_scaled
0	female	group B	bachelor's degree	standard	none	72.0	72.0	74.0	0.654321	0.636364	0.654321
1	female	group C	some college	standard	completed	69.0	90.0	88.0	0.617284	0.870130	0.854321
2	female	group B	master's degree	standard	none	90.0	95.0	93.0	0.876543	0.935065	0.904321

Next steps:

[Generate code with df](#)
[New interactive sheet](#)

```
df[['math_scaled', 'reading_scaled', 'writing_scaled']].describe()
```

	math_scaled	reading_scaled	writing_scaled
count	985.000000	985.000000	985.000000
mean	0.584496	0.602386	0.626169
std	0.183006	0.186100	0.175328
min	0.000000	0.000000	0.000000
25%	0.469136	0.480519	0.505882
50%	0.580247	0.610390	0.635294
75%	0.716049	0.727273	0.752941
max	1.000000	1.000000	1.000000



